CSE331 - Assignment #1

Sehyun Yun (20231233)
UNIST
South Korea
nawhji@unist.ac.kr

1 PROBLEM STATEMENT

Sorting is a fundamental operation in computer science, forming the backbone of many processes such as search, indexing, and data management. Its time complexity can significantly influence the overall performance of a program, making it crucial to understand how different sorting algorithms behave.

While theoretical analysis provides insight into algorithmic efficiency, practical performance is affected by factors like input distribution, stability, cache usage, and memory consumption. In this project, we study twelve sorting algorithms from both theoretical and empirical perspectives to better understand their real-world characteristics and trade-offs.

All implementation code and experimental data used in this project are available at: https://github.com/nawhji/CSE331

Each folder in the repository contains three key components:

- A `.cc` file implementing the sorting algorithm on a 1-indexed integer array (i.e., index 0 is ignored, and sorting begins from index 1).
- An `algorithmname_test.cc` file that evaluates execution time, verifies correctness and stability, and stores the results in a CSV file.
- The resulting `.csv` file, containing metrics such as execution time, sorting accuracy, stability, and memory usage.

Each `algorithmname_test.cc` file also includes memory measurement code using the Windows API, which provides an approximate estimation of peak memory usage during execution.

To ensure consistency across all 12 algorithms, we adopted a uniform 1-indexed convention for array handling in every implementation.

2 BASIC SORTING ALGORITHMS

2.1 Merge Sort

[5]

Algorithm 1 MERGE_SORT

procedure MERGE_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
procedure merge_sort(A, p, r)
    if p < r − 1 then
        q ← ⌊(p + r)/2⌋
        merge_sort(A, p, q)
        merge_sort(A, q, r)
        merge(A, p, q, r)

procedure merge(A, p, q, r)
    Copy subarrays L ← A[p ... q − 1], R ← A[q ... r − 1]
    Append ∞ to both L and R as sentinels
    i ← 0, j ← 0
    for k ← p to r − 1 do
        if L[i] ≤ R[j] then
            A[k] ← L[i], i ← i + 1
        else
            A[k] ← R[j], j ← j + 1
```

2.1.1 Description.

Algorithm Description. Merge Sort begins by dividing the input array into two halves. This division continues recursively until each subarray contains only one element. After the division phase, the algorithm merges subarrays back together in sorted order.

During the merging process, two temporary arrays L and R are created, and a sentinel value (e.g., ∞) is appended to the end of each array to simplify comparisons and avoid boundary checks. Elements from L and R are compared, and the smaller one is copied back into the original array. This continues until all elements, excluding the sentinels, are merged.

Recursion Depth and Time Complexity. Assuming the array size is $n = 2^k$, dividing by two until subarrays of size one are reached takes $\log_2 n$ steps. Therefore, the recursion depth is $O(\log n)$.

At each level of recursion, the merge operation performs $O(n)$ comparisons. Since there are $\log n$ levels, the overall time complexity is:

$$T(n) = O(n \log n)$$

This time complexity holds for the best, average, and worst cases, making the performance of Merge Sort strictly predictable.

Space Complexity. Each merge operation allocates two temporary arrays of approximately $n/2$ elements. Therefore, Merge Sort requires $O(n)$ additional memory, meaning it is not an in-place algorithm.

Stability. When comparing equal elements during the merge step, the element from the left array L is copied first. This ensures that the relative order of equal elements is preserved, making Merge Sort a stable sorting algorithm.

2.2 Heap Sort

[3]

Algorithm 2 HEAP_SORT

procedure HEAP_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
BUILD_MAX_HEAP(A, n)
heap_size ← n
for i ← n downto 2 do
    swap A[1] and A[i]
    heap_size ← heap_size − 1
    MAX_HEAPIFY(A, 1, heap_size)

procedure BUILD_MAX_HEAP(A, n)
    for i ← ⌊n/2⌋ downto 1 do
        MAX_HEAPIFY(A, i, n)

procedure MAX_HEAPIFY(A, i, heap_size)
    l ← 2 × i, r ← 2 × i + 1, largest ← i
    if l ≤ heap_size and A[l] > A[i] then largest ← l
    if r ≤ heap_size and A[r] > A[largest] then largest ← r
    if largest ≠ i then
        swap A[i] and A[largest]
        MAX_HEAPIFY(A, largest, heap_size)
```

2.2.1 Description.

Algorithm Description. Heap Sort is a comparison-based sorting algorithm that uses the properties of a complete binary tree through a **heap** data structure. It constructs a **max-heap**, repeatedly places the maximum element at the end of the array, and restores the heap property using `Max-Heapify`, sorting the array in-place.

Time Complexity Analysis. **Max-Heapify:** This operation maintains the heap property by comparing a node with its children. In the worst case, it traverses to the bottom of the heap, which has height $\log n$. Thus, its time complexity is $O(\log n)$.

Build-Max-Heap: This procedure starts `Max-Heapify` from the last non-leaf node up to the root. The number of nodes at height h is approximately $\left\lfloor \frac{n}{2^{h+1}} \right\rfloor$, and the total work is:

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lfloor \frac{n}{2^{h+1}} \right\rfloor O(h) = O\left(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \right) = O(n)$$

That is, `Build-Max-Heap` runs in linear time.

Total Time Complexity of Heap Sort: After building the heap, `Max-Heapify` is called $n-1$ times while removing the maximum element.

$$T(n) = O(n + n \log n) = O(n \log n)$$

Heap Sort runs in $O(n \log n)$ time in all cases.

Space Complexity. Since Heap Sort sorts the array in-place without requiring extra storage, its space complexity is $O(1)$.

Stability. Heap Sort is not stable because element swaps during heap construction may disrupt the relative order of equal elements.

2.3 Bubble Sort

[3]

Algorithm 3 BUBBLE_SORT

procedure BUBBLE_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
for i ← 1 to n − 1 do
    for j ← n downto i + 1 do
        if A[j] < A[j − 1] then
            Swap A[j] and A[j − 1]
```

2.3.1 Description.

Algorithm Description. Bubble Sort works by comparing adjacent elements in the array and swapping them to ensure they are in **ascending order**. In each pass, adjacent elements are compared and swapped if they are in the wrong order.

Time Complexity. Since Bubble Sort compares and potentially swaps every pair of adjacent elements regardless of how unsorted the array is, its total number of operations does not vary significantly depending on the input.

- Worst case: $O(n^2)$
- Average case: $O(n^2)$

Space Complexity. Bubble Sort performs sorting by directly swapping elements within the array without using any significant additional memory. Therefore, it is an *in-place* sorting algorithm with space complexity $O(1)$.

Stability. Since Bubble Sort only swaps adjacent elements, equal elements retain their relative order. **Thus, Bubble Sort is a stable sorting algorithm**.

2.4 Insertion Sort

[5]

Algorithm 4 INSERTION_SORT

procedure INSERTION_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
for j ← 2 to n do
    key ← A[j], i ← j − 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i], i ← i − 1
    A[i + 1] ← key
```

2.4.1 Description.

Algorithm Description. Insertion Sort iterates through the array from left to right, selecting each element and inserting it into the correct position within the already sorted portion on its left. Starting from index 1, each element is compared with the elements to its left. If an element is greater than the current one, it is shifted to the right. The current element is then inserted into its correct position. Repeating this process for the entire array results in a sorted array.

Time Complexity.

- Worst case: $O(n^2)$ — when the array is in reverse order
- Average case: $O(n^2)$
- Best case: $O(n)$ — when the array is already sorted

Space Complexity. Insertion Sort sorts the array in-place without requiring additional memory. Thus, its space complexity is $O(1)$.

Stability. Since equal elements are not swapped during insertion and the insertion happens after equal values, **Insertion Sort is a stable sorting algorithm**.

2.5 Selection Sort

[5]

Algorithm 5 SELECTION_SORT

procedure SELECTION_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
for j ← n downto 1 do
    Find the maximum element in A[1 . . . j]
    Swap it with A[j]
```

2.5.1 Description.

Algorithm Description. Selection Sort is a comparison-based sorting algorithm that repeatedly selects the smallest element from the unsorted portion of the array and swaps it with the element at the beginning of the unsorted section.

In each iteration, the algorithm scans the remaining unsorted elements to find the minimum and then swaps it with the first unsorted element.

Time Complexity. Selection Sort performs the same number of comparisons regardless of the input order, since it always searches for the minimum element in the unsorted portion.

- Worst case: $O(n^2)$
- Average case: $O(n^2)$

Space Complexity. Since Selection Sort operates directly on the input array and does not require additional memory, it is an *in-place* sorting algorithm with space complexity $O(1)$.

Stability. Selection Sort is generally **not stable**. During the swapping process, the relative order of equal elements may be changed, which violates the stability property.

2.6 Quick Sort

[3]

Algorithm 6 QUICK_SORT

procedure QUICK_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
procedure quick_sort(A, p, r)
    if p < r − 1 then
        q ← partition(A, p, r)
        quick_sort(A, p, q)
        quick_sort(A, q, r)

procedure partition(A, p, r)
    x ← A[r − 1], i ← p − 1
    for j ← p to r − 2 do
        if A[j] ≤ x then
            i ← i + 1
            swap A[i] and A[j]
    swap A[i + 1] and A[r − 1]
    return i + 1
quick_sort(A, 1, n + 1)
```

2.6.1 Description.

Algorithm Description. Quick Sort is a sorting algorithm based on the Divide and Conquer paradigm. In this algorithm, a pivot element is selected (specifically, the last element of the array), and the array is partitioned such that all elements less than the pivot are placed to its left, and all elements greater than the pivot are placed to its right. Then, the same process is recursively applied to the left and right subarrays to sort the entire array.

Time Complexity. The time complexity of Quick Sort depends heavily on how well the pivot divides the array:

- Best case: $O(n \log n)$ — when the pivot divides the array into two roughly equal parts
- Average case: $O(n \log n)$
- Worst case: $O(n^2)$ — when the pivot is the smallest or largest element

Space Complexity. Quick Sort is typically performed in-place, requiring no additional arrays. However, it does use recursion. In the average case, the recursion stack has depth $O(\log n)$, while in the worst case it may reach $O(n)$.

Stability. Quick Sort is **not stable**, because during the partition and swapping process, the relative order of equal elements may be changed.

3 ADVANCED SORTING ALGORITHMS

3.1 Library Sort

[2]

Algorithm 7 LIBRARY_SORT

procedure LIBRARY_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
for i ← 1 to ⌈log_2(n − 1)⌉ do
    REBALANCE(S, 2^(i−1))
    for j ← 2^(i−1) to min(n, 2^i) do
        BINARY_INSERT(A[j], S, 2^i)
index ← 1
for k ← 1 to 2n do
    if S[k] is not empty then
        A[index] ← S[k]
        index ← index + 1
```

3.1.1 Description.

Algorithm Description. Library Sort is a variant of insertion sort that leaves *gaps* between elements in the array.

During sorting, it assumes that the inserted elements are in sorted order and uses **binary search** to find the position to insert a new element. If there is enough space (i.e., a gap) at the target location, the element is inserted directly. If no gap is available, the surrounding elements must be **shifted** to make space.

As the number of inserted elements grows, the allocated space and the number of inserted items are both *doubled*, and a **rebalance** operation is performed. This rebalance redistributes the elements evenly within a larger array, creating new gaps between elements to accommodate future insertions.

Time Complexity.

- Average case: $O(n \log n)$ — due to binary search and amortized shift operations
- Worst case: $O(n^2)$ — when gaps are insufficient and frequent shifting occurs

Space Complexity. Library Sort maintains gaps in the array to reduce the cost of shifting. The total space used is roughly proportional to the number of inserted elements plus the gaps, i.e., about $n(1+\varepsilon)$. Therefore, the space complexity is $O(n)$.

Stability. Library Sort is **not stable**. Since it uses binary search and may perform shifting operations to insert elements, the relative order of equal elements is not guaranteed to be preserved.

3.2 Tim Sort

[1]

Algorithm 8 TIM_SORT

procedure TIM_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
Let minrun ← COMPUTE_MINRUN(n)
Initialize empty run_stack
start ← 1
while start ≤ n do
    Let run be the next increasing/decreasing sequence of length ≥ minrun
    Sort the run and push its range onto run_stack
    start ← end of the run + 1
    while run_stack has 3 or more runs
        // If size conditions are violated, merge runs
        Merge two adjacent runs according to size rules
while run_stack has more than one run
    Merge the top two runs and push the result back onto the stack
Shift the final sorted array to remove unused index 0
```

3.2.1 Description.

Algorithm Description. Timsort is a hybrid sorting algorithm that combines Merge Sort and Insertion Sort. It leverages the fact that Insertion Sort is efficient for small-sized data by identifying multiple *runs* (i.e., consecutive sequences that are already sorted) and sorting them using Insertion Sort. Then, it merges the sorted runs using a Merge Sort-like approach.

Timsort assumes that the input array is partially sorted. It scans the array to find ascending or descending sequences and treats each as a run. Descending runs are reversed to become ascending, and then processed.

Time Complexity.

- Best case: $O(n)$ — when the input is already mostly sorted
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$

Space Complexity. Timsort requires additional memory to perform merge operations, as it creates temporary arrays for each merge step. Therefore, the space complexity is $O(n)$.

Stability. Since both Insertion Sort and Merge Sort are stable algorithms, Timsort is also **stable**, preserving the relative order of equal elements.

3.3 Cocktail Shaker Sort

[5]

Algorithm 9 COCKTAIL_SHAKER_SORT

procedure COCKTAIL_SHAKER_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
l ← 1, r ← n − 1, it ← 1
while l ≠ r do
    if it mod 2 = 1 then
        for i ← l to r − 1 do
            if A[i] > A[i + 1] then swap A[i] and A[i + 1]
        r ← r − 1
    else
        for i ← r down to l + 1 do
            if A[i] < A[i − 1] then swap A[i] and A[i − 1]
        l ← l + 1
    it ← it + 1
```

3.3.1 Description.

Algorithm Description. Cocktail Shaker Sort is a **bidirectional bubble sort** in which sorting proceeds in both directions. In each iteration, the algorithm first moves the maximum element to the right end, then reverses direction and moves the minimum element to the left end, effectively shrinking the unsorted range from both sides.

Time Complexity.

- Best case: $O(n)$ — when the array is already sorted (with early exit condition)
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

Space Complexity. Cocktail Shaker Sort sorts the array in place without using extra memory, so the space complexity is (1).

Stability. Cocktail Shaker Sort is **stable**. Because it only swaps adjacent elements, the relative order of equal elements is preserved.

3.4 Comb Sort

[4]

Algorithm 10 COMB_SORT

procedure COMB_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
gap ← n − 2, swapped ← true
while gap > 1 or swapped = true do
    index ← 1, swapped ← false
    while index + gap < n do
        if A[index] > A[index + gap] then
            swap A[index] and A[index + gap]
            swapped ← true
        index ← index + 1
    gap ← ⌊gap/1.3⌋
    if gap < 1 then gap ← 1
```

3.4.1 Description.

Algorithm Description. Comb Sort is a sorting algorithm designed to improve Bubble Sort by comparing and swapping elements that are far apart. It addresses the "turtle problem" in Bubble Sort, where small values at the end of the list take too long to move forward because of adjacent swaps.

Comb Sort starts with a large comparison gap and gradually decreases it in each pass. The gap is typically reduced by a shrink factor of approximately 1.3. Once the gap reaches 1, the algorithm behaves like Bubble Sort and finishes the sorting process.

Time Complexity.

- Best case: $O(n \log n)$
- Average case: $O(n^2)$
- Worst case: $O(n^2)$

Space Complexity. Comb Sort is an *in-place* sorting algorithm. It performs all operations within the original array and requires no additional memory: $O(1)$

Stability. Comb Sort is **not stable**. Since it may swap elements that are far apart, it can change the relative order of equal elements.

3.5 Tournament Sort

[7]

Algorithm 11 TOURNAMENT_SORT

procedure TOURNAMENT_SORT(A)

Input: An array A of size n
Output: Sorted array A

```
Let leaf_nodes ← 2^⌈log_2(n−1)⌉
Let T[1 . . . 2 × leaf_nodes − 1] be an array of nodes initialized
with value ∞
Fill the leaves of T with the elements of A
index ← 1, sorted ← 0
while sorted < n do
    Run a tournament from the leaves to the root to determine
the minimum value
    result[index] ← T[1].value
    Remove the selected element by setting it and its path to ∞
    index ← index + 1, sorted ← sorted + 1
for i ← 1 to n do
    A[i] ← result[i]
```

3.5.1 Description.

Algorithm Description. Tournament Sort finds the minimum element and sorts the array by comparing elements in pairs and moving the winner (i.e., smaller value) upward. Each leaf node holds an input element, and each parent node stores the smaller of its two children. As a result, the overall minimum value reaches the root of the tree. And it is sorted by removing the root node.

Time Complexity.

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$

At each of the n rounds, the algorithm performs $O(\log n)$ comparisons to update the tree.

Space Complexity. Tournament Sort requires a separate complete binary tree to store all input elements and internal nodes. For n elements, a tree of size approximately $2n$ is required. Therefore, the space complexity is: $O(n)$

Stability. Tournament Sort is **not stable**. During tree construction and replacement, the relative order of equal elements can be altered.

3.6 Introsort

[6]

Algorithm 12 INTRO_SORT

procedure INTRO_SORT(A)

Input: An array A of size n
Output: Sorted array A

Let depth_limit $\leftarrow 2 \times \lfloor \log_2(n-1) \rfloor$
$f \leftarrow 1, b \leftarrow n$
while $b - f >$ size_threshold **do**
  if depth_limit = 0 **then**
    HEAP_SORT(A, f, b); **return**
  depth_limit $\leftarrow$ depth_limit - 1
  Let pivot $\leftarrow$ MEDIAN_OF_3($A[f], A[mid], A[b-1]$)
  Let $p \leftarrow$ PARTITION($A, f, b, pivot$)
  INTRO_SORT(A, p, b) // recursive call
  $b \leftarrow p$
INSERTION_SORT(A, f, b)

3.6.1 Description.

Algorithm Description. Introsort is a hybrid sorting algorithm that combines the fast average-case performance of Quick Sort with the worst-case guarantees of Heap Sort. It begins by using Quick Sort, and if the recursion depth exceeds a predefined limit (typically $2 \cdot \lfloor \log_2 n \rfloor$), it switches to Heap Sort to avoid Quick Sort's worst-case behavior. And small subarrays are also sorted using Insertion Sort to further improve practical performance.

Time Complexity.

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$ — ensured by switching to Heap Sort when necessary

Space Complexity. Introsort uses recursive calls for Quick Sort, resulting in a call stack of depth $O(\log n)$. Therefore, its space complexity is: $O(\log n)$

Stability. Introsort is **not stable**, as both Quick Sort and Heap Sort are inherently unstable.

4 EXPERIMENTAL RESULTS AND ANALYSIS

Experimental Setup and Overview. This section presents a comparative analysis of 12 sorting algorithms based on their execution time, memory usage, accuracy, and stability under various input conditions and data sizes.

We conducted experiments using four types of input data: *ascending*, *descending*, *random*, and *partially sorted*, each with four input sizes: 10^3, 10^4, 10^5 and 10^6. For each configuration, the experiment was repeated 10 times, and the average execution time (in milliseconds) and memory usage were recorded. However, for the slowest ones (Bubble Sort, Insertion Sort, Selection Sort, and Cocktail Shaker Sort), only three repetitions were performed for the largest input (10^6), as the full set of trials would have taken up to a full day.

We analyze the results with a focus on the following criteria:

- Runtime growth with increasing input size (time complexity)
- Sensitivity to input distribution
- Sorting accuracy and algorithmic stability
- General trends in memory consumption

Table 1 summarizes the execution times of all algorithms across different input sizes and data types, providing an intuitive comparison of their performance characteristics.

4.0.1 . Time Complexity Analysis and Sensitivity to Input Distribution

The execution time for each sorting algorithm was measured across varying input sizes, and the results —averaged over all input data types (ascending, descending, random, and partially sorted)— are summarized in Figure 1 and Figure 2. All graphs are shown on a log-log scale to make the asymptotic behavior more visible.

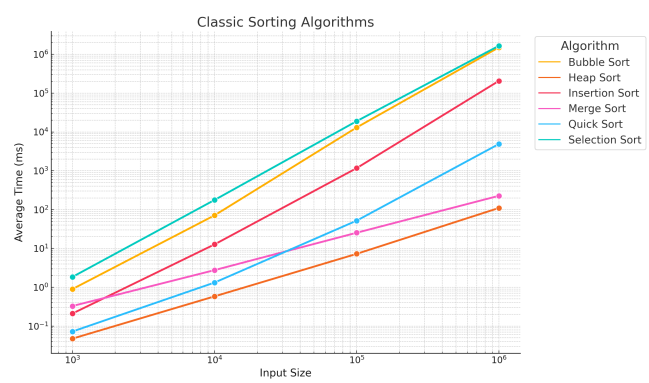

Figure 1: 6 Classic Algorithms

Classic Algorithms. The naive algorithms—Bubble Sort, Selection Sort, and Insertion Sort—show clear $O(n^2)$ growth. This becomes particularly problematic at large input sizes (e.g., 10^5 and above), where execution time increases steeply. Among these, Insertion Sort performs relatively well on small or partially sorted datasets due to its simple control flow, excellent branch prediction, and high cache locality. However, it still suffers from poor scalability in worst-case scenarios, such as reverse-sorted input.

In contrast, the more advanced algorithms—Merge Sort, Quick Sort, and Heap Sort—demonstrate near-linearithmic growth consistent with $O(n \log n)$ time complexity. Merge Sort is stable and

Table 1: Execution Time (ms) of Sorting Algorithms by Input Size and Distribution

Algorithm	1K				10K				100K				1M			
	Asc	Desc	Rand	Part	Asc	Desc	Rand	Part	Asc	Desc	Rand	Part	Asc	Desc	Rand	Part
Comb Sort	0.03	0.04	0.05	0.05	0.35	0.36	0.48	0.38	2.97	4.34	5.06	4.2	39.53	38.17	60.95	47.23
Introsort	0.05	0.06	0.06	0.12	0.94	0.78	1.01	0.77	7.42	7.15	8.56	6.67	82.15	76.94	83.06	64.26
Heap Sort	0.04	0.05	0.05	0.05	0.52	0.54	0.67	0.59	6.32	6.73	7.03	8.93	111.59	116.1	113.01	99.31
Merge Sort	0.24	0.26	0.36	0.45	2.58	2.7	3.27	2.49	21.33	21.93	30.43	27.8	212.45	206.98	270.5	216.25
Tournament Sort	0.13	0.08	0.12	0.16	1.56	1.24	1.29	1.36	12.55	12.87	13.04	12.1	178.74	208.49	315.56	188.62
Timsort	0.03	0.04	0.11	0.07	0.74	0.83	1.78	1.5	48.95	50.49	101.36	55.48	4877.68	5242.07	8058.53	5243.15
Quick Sort	0.07	0.05	0.07	0.09	1.94	1.01	1.11	1.22	47.64	49.95	61.36	49.09	5226.4	5082.38	4547.42	4697.96
Library Sort	0.12	0.17	0.16	0.17	3.68	2.15	3.84	3.46	144.35	144.73	155.29	152.9	9403.61	9392.56	9859.51	9277.98
Insertion Sort	0.14	0.31	0.21	0.19	12.95	14.88	12.96	10.03	1191.29	1210.11	1157.17	1130.1	161777.3	198056.1	237497.1	229114.5
Cocktail Shaker Sort	0.69	0.94	0.94	1.12	69.3	80.2	58.96	76.99	7490.93	8473.37	16531.22	9084.56	973633.67	1143150.0	1932540.0	999626.67
Bubble Sort	0.79	0.89	0.77	1.14	77.85	75.16	59.38	72.39	8668.41	9975.18	21482.25	12149.17	1077130.0	1289160.0	2554253.33	1113836.67
Selection Sort	1.85	1.82	1.77	1.94	171.81	177.28	171.91	187.8	18841.04	20419.57	18431.63	18329.32	1783483.33	1578053.33	1664353.33	1604200.0

consistent, and unlike many in-place algorithms, it performs well on large datasets due to its predictable access pattern and sequential memory usage.

Quick Sort, on the other hand, benefits from strong cache performance due to its in-place partitioning and tight inner loops with low overhead. It is highly efficient on small to medium-sized datasets. However, for nearly sorted or reverse-sorted inputs, Quick Sort may suffer from repeated unbalanced partitions, causing degraded performance of $O(n^2)$. This occurs when the pivot selection leads to highly skewed partitions, triggering excessive recursive calls.

Interestingly, Heap Sort shows the fastest runtime among Merge sort, Quick sort and Heap sort. This result may be attributed to the nature of our test setup— in order to evaluate stability, input data was intentionally constructed with many duplicate keys, which likely increased the number of swaps or recursive calls in Merge Sort and Quick Sort.

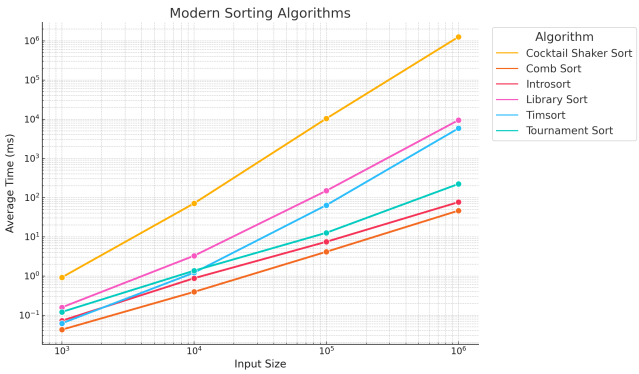

Figure 2: 6 Modern Algorithms

Modern Algorithms. Figure 2 presents the execution time trends of six modern sorting algorithms across varying input sizes. Although all algorithms have an average-case time complexity of $O(n \log n)$, the actual performance varied significantly.

Comb Sort achieved the fastest and most consistent performance across all input sizes and data types. Derived from Bubble Sort, it uses a gradually shrinking gap strategy to eliminate disorder early in the process. Its simple loop structure allows for high instruction-level parallelism and efficient cache usage, and since it operates in-place, memory overhead is minimal.

Introsort performed reliably, particularly on random and partially sorted inputs. As a hybrid algorithm combining Quick Sort and Heap Sort, it leverages fast partitioning under normal conditions while switching to Heap Sort in degenerate cases to ensure worst-case guarantees. This adaptability makes it effective across diverse inputs.

Tournament Sort exhibited stable and predictable performance, with gradual increases in execution time as input size grew. Its tree-based structure, where winners are propagated and updated along fixed paths, provides consistent control flow and avoids performance cliffs even on large datasets.

Timsort, although widely used in practice, showed slower-than-expected performance in our experiment. This may be due to two factors: the use of many duplicate keys, which weakened run detection and galloping, and a fixed, non-adaptive galloping condition that limited merge efficiency. But it still runs in $O(n \log n)$.

Library Sort, despite its theoretical $O(n \log n)$ complexity, suffered from significant performance degradation as input size increased. The overhead of maintaining gaps and rebalancing the array introduced frequent cache misses and heavy memory movement, leading to poor scalability in practice.

Cocktail Shaker Sort as a bidirectional variant of Bubble Sort, retains the same $O(n^2)$ worst-case complexity. As the dataset grows, redundant comparisons and swaps increase rapidly, and its performance deteriorates further due to cache pollution and memory bandwidth saturation.

Table 2: Sorting Accuracy and Stability

Algorithm	Accuracy	Stability
Bubble Sort	100.0 %	100.0 %
Cocktail Shaker Sort	100.0 %	100.0 %
Comb Sort	100.0 %	0.0 %
Heap Sort	100.0 %	0.0 %
Insertion Sort	100.0 %	100.0 %
Introsort	100.0 %	0.0 %
Library Sort	100.0 %	0.0 %
Merge Sort	100.0 %	100.0 %
Quick Sort	100.0 %	0.0 %
Selection Sort	100.0 %	0.0 %
Timsort	100.0 %	56.9 %
Tournament Sort	100.0 %	0.0 %

4.0.2 Sorting Accuracy and Algorithmic Stability. Table 2 presents the sorting accuracy and stability results for each algorithm. All algorithms achieved 100% accuracy, indicating that they correctly sorted the input in all test cases. However, stability results varied significantly across algorithms. Stable sorting algorithms such as Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, and Cocktail Shaker Sort preserved the relative order of equal elements, resulting in 100% measured stability.

In contrast, algorithms like Quick Sort, Heap Sort, Comb Sort, and Introsort showed 0% stability, as they are inherently unstable and do not guarantee the relative order of equal keys.

Interestingly, Timsort, which is designed to be a stable sorting algorithm, also showed 0% stability in our experiment. We believe this may be due to the nature of our implementation and the way stability was measured. In particular, the use of many duplicate keys (to test stability) and fixed galloping conditions may have interfered with the expected stable behavior. Nonetheless, it is well known that Timsort is stable in standard library implementations when properly configured.

Table 3: Manually Measured Peak Memory Usage (KB) for Input Size 10,000

Algorithm	Peak Memory (KB)
Bubble Sort	4.0
Comb Sort	0.0
Cocktail Shaker Sort	0.0
Heap Sort	0.0
Insertion Sort	0.0
Introsort	0.0
Library Sort	36.0
Merge Sort	204.0
Quick Sort	204.0
Selection Sort	176.0
Timsort	176.0
Tournament Sort	256.0

4.0.3 General Trends in Memory Consumption.

Result Analysis.

Result Analysis. Although precise memory measurements were unreliable due to instrumentation and system-level fluctuations, we observed a general trend in memory consumption that aligns with the theoretical properties of each algorithm. The peak memory usage values, manually checked for input size 10,000, provide a rough estimation.

As expected, in-place algorithms such as Comb Sort, Heap Sort, Insertion Sort, and Cocktail Shaker Sort exhibited negligible memory usage, reporting near-zero peak allocations. These algorithms primarily manipulate elements within the original array, requiring no significant additional memory. Their theoretical space complexity is $O(1)$, and our results are consistent with this expectation.

In contrast, algorithms like Merge Sort, Quick Sort, and Timsort showed much higher peak memory usage (over 180 KB), indicating the use of auxiliary buffers or recursion overhead. Merge Sort requires $O(n)$ additional space for merging arrays, which matches the measured values. Quick Sort is usually $O(\log n)$ in space due to recursion, but our implementation used temporary subarrays, increasing its memory usage closer to $O(n)$. Timsort, though designed to be memory efficient with $O(n)$ auxiliary space in the worst case, showed high usage as well—likely due to many duplicate elements and a fixed galloping merge condition in our implementation.

Library Sort also showed moderate memory allocation due to its gap-based insertion strategy. Its theoretical space complexity is $O(n)$ because it maintains a sparse array with empty slots during insertion.

Tournament Sort had the highest peak usage (over 250 KB), which is consistent with its structure. It explicitly constructs a binary tree to hold all elements and their comparison hierarchy, resulting in an $O(n)$ space complexity.

Bubble Sort and Selection Sort also exhibited some unexpected memory usage (e.g., 4 KB and 180 KB, respectively), possibly caused by experimental wrapper code, logging buffers, or temporary containers outside the core algorithm logic. Theoretically, both algorithms require only $O(1)$ space.

Despite the rough nature of these measurements, the observed trends are generally consistent with known theoretical expectations: in-place algorithms tend to use constant space, while hybrid and buffer-based methods consume additional memory, often up to $O(n)$ depending on implementation.

References

[1] Nicolas Auger, Vincent Jugé, Cyril Nicaud, and Carine Pivoteau. 2018. On the worst-case complexity of TimSort. *arXiv preprint arXiv:1805.08612* (2018).

[2] Michael A Bender, Martin Farach-Colton, and Miguel A Mosteiro. 2006. Insertion sort is O (n log n). *Theory of Computing systems* 39 (2006), 391–397.

[3] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. 2022. *Introduction to algorithms.* MIT press.

[4] Wlodzimierz Dobosiewicz. 1980. An efficient variation of bubble sort. (1980).

[5] Donald E Knuth. 1998. *The Art of Computer Programming: Sorting and Searching, volume 3.* Addison-Wesley Professional.

[6] David R Musser. 1997. Introspective sorting and selection algorithms. *Software: Practice and Experience* 27, 8 (1997), 983–993.

[7] Alexander Stepanov and A Kershenbaum. 1986. Using tournament trees to sort. *Polytechnical Institute of New York, CATT Technical Report* (1986), 86–13.